CG3201 Machine Learning and Deep Learning
Lab 2 – Bayesian Learning and Modelling

1. Task Introduction and Dataset

In this lab, we will explore how to predict if an email is spam or ham from two different perspectives, Bayesian decision theory and Frequentist decision theory.

Before we build such models, we need to understand the data. Our dataset contains thousands of emails with two labels, 1 for spam and 0 for ham. For each email, we have the frequency of all the words from the vocabulary which appear in the email. Please refer to the emails.csv file to better understand the dataset.

Data loading and splitting are implemented in prepare_train_test() function in utils.py and the function will return training, testing data (in numpy) as well as the vocabulary.

2. Bayesian decision theory

2.1 Training

Recall that in Bayesian decision theory we have the expected posterior loss for each action as follows:

$$\rho(a, x) = \sum_{h \in H} L(a, h) P(h|x)$$

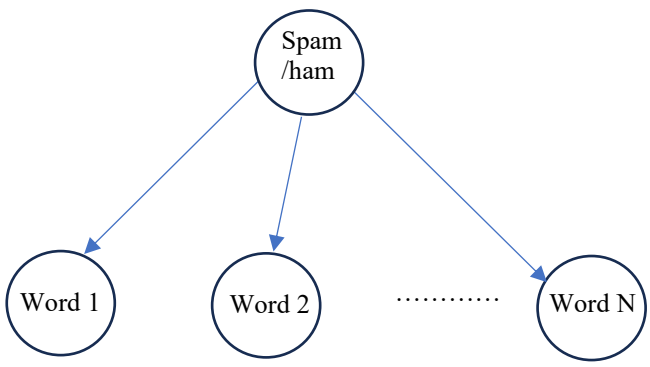
For classification with zero-one loss, the expected posterior loss becomes:

$$\rho(a, x) = 1 - P(\hat{y} = y^*|x)$$

where $\hat{y}$ is the predicted label and y^* is the true label. Essentially, our prediction becomes choosing the label with the highest probability.

$$\pi(x) = \arg\max_{\hat{y} \in \mathcal{Y}} P(\hat{y}|x)$$

We will use Naïve Bayes to solve this classification task by utilizing this graphical model below:



Here we are making a very simple assumption; all the words are conditionally independent on the label. In real life, the words are highly related but this naïve bayes model works well in this task despite its simplicity. You can now find $P(spam|words)$, $P(ham|words)$ using Bayes rule:

$$1)\ P(\text{spam} \mid \text{words}) \propto P(\text{spam}) \prod_{i=1}^{n} P(\text{word}_i \mid \text{spam})$$

$$2)\ P(\text{ham} \mid \text{words}) \propto P(\text{ham}) \prod_{i=1}^{n} P(\text{word}_i \mid \text{ham})$$

We can obtain prior $P(\text{spam})$, P(ham) and the likelihood $P(\text{word}_i \mid \text{spam})$, $P(\text{word}_i \mid \text{ham})$ from the training dataset. To avoid the likelihood of becoming 0 and destroying the whole chain of products, we will apply Laplacian smoothing. Laplacian smoothing assumes each word in the vocabulary appears once for each label before we see any data. Hence likelihood becomes:

$$P(\text{word}_i \mid \text{label}) = \frac{\text{count}(\text{word}_i,\text{label}) + 1}{\text{count}(\text{label}) + |V|}$$

where $|V|$ is the size of the vocabulary.

Todo: Finish fit method from NaiveBayes class.

Now we are ready for inference!

2.2 Testing

Using zero-one loss, make the predictions for the test data and do the following:
1) Report the accuracy, precision, recall and f1-score of the model for zero-one loss.
2) Plot the Precision Recall Curve to see the classification power of the model.

To prevent underflow when computing a long chain of product probability, you need to use log probability.

Todo: Finish the rest of naïve_bayes.py and utils.py

3. Frequentist decision theory

3.1 Data Preprocessing

Before diving into training, let do some data preprocessing. For each email, instead of current bag of words approach (list of frequencies for all words in the vocabulary), we will use term frequency – inverse document frequency (TF-IDF). The aim is to give higher weight to signature words which appear frequently in the current email but do not appear in other emails and lower the importance of frequent words like “the” which appears in a lot of emails. This is the formula for TF-IDF:

$$\text{tf-idf}(t) = \text{tf}(\text{t, d}) * \text{idf}(\text{t}) = \text{tf}(\text{t, d}) * (\ln\left(\frac{1+n}{1+\text{df}(t)}\right) + 1)$$

where t is the term, we are considering, tf(t,d) counts how many times t appears in email d, n is the number of emails and df(t) is the number of emails containing term t. You can then use the training data to compute tf-idf for every word in the vocabulary. Now, for each email coming in, you get tf-idf for every word and normalize the score across the words, so they sum up to 1.

Todo: Finish the TFIDF class.

3.2 Training

In this part, we will train SVM using empirical risk minimization and use cross validation to choose the best parameter C. This is the regularized empirical risk minimization:

$$\widehat{f_{RERM}} = \arg\min_f \left[\frac{1}{N}\sum_{n=1}^{N} \mathrm{l}\big(y_n, f(x_n)\big) + \lambda C(f)\right]$$

We can use supervised learning where we find the predictor which minimizes the risk of the training data. Recall we have the loss function for SVM:

$$L = \frac{1}{2}\|\boldsymbol{w}\| + C\sum_i \max\big(0, 1 - y_i(\boldsymbol{w}^T\boldsymbol{x_i} + b)\big)$$

Naturally, the margin acts as a regularizer. To speed up the training, you should use mini-batch gradient descent with batch size of 64 and use NumPy vectorization to update the parameters. Moreover, to make training more stable for larger C where the loss function maybe steeper due to higher penalty for violations, use learning rate decay in your training.

Hint: As C becomes larger, the loss landscape will be steeper. Draw the training curve across epochs to adjust your learning rate decay strategy accordingly.

Todo: Finish the fit function from SVM class.

3.3 Cross validation

We have one hyper-parameter C to choose, and we can choose the best value for C with cross validation because our dataset is small. In this part, use 4 folds cross validation to choose C which leads to highest average f1-score.

Todo: Finish the rest of SVM class and do cross validation inside main from svm.py

3.4 Testing

Retrain the whole training set using best C we just found, make the predictions for the test data and report the accuracy, precision, recall and f1-score of the model.

Todo: Finish main in svm.py

4. Reflections

1) Can you guess why Naïve Bayes, despite its simplicity, perform well?
2) Can you guess why SVM also perform well?

END